Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 00 (2025) 000–000

Procedia Computer Science

www.elsevier.com/locate/procedia

International Conference on Machine Learning and Data Engineering

Dropout-Aware Training for Sensor-Robust Fault Localization with Spatio-Temporal Graph Neural Networks

P K Sangameswar[a], M Anbazhagan[a,*]

[a]*Department of Computer Science and Engineering, Amrita School of Computing, Coimbatore, Amrita Vishwa Vidyapeetham, India*

Abstract

Spatio-temporal graph neural networks (STGNNs) localize faults accurately in distribution grids instrumented with only a few micro-phasor measurement units (μPMUs), but their evaluations assume every sensor always reports, which deployed systems do not guarantee. Following the STGNN methodology of Karabulut et al. (2026) on the IEEE 123-bus feeder, we train five architectures on our own dataset, the strongest reaching a soft-voting macro-F1 of 0.85, and present what is to our knowledge the first systematic evaluation of STGNN robustness to μPMU loss, dropping $k \in \{0, 1, 2, 5, 10\}$ of 25 sensors at inference. Across the five architectures evaluated here, clean accuracy did not predict robustness: the most accurate one degraded the fastest, falling below 0.10 at 40% loss. The inversion holds in all three seeds but does not reach significance at $n = 3$, so we report it as a consistent observation rather than an established ordering. Dropout-aware training, which masks measurement nodes during training, raises retention at 40% loss from 5.7% to 80.3% on that backbone ($p < 0.01$, paired over three training seeds) at measured parity in inference latency and parameter count, and improves all five architectures. Notably, an availability-gated architectural variant only matches it, indicating that sensor-dropout robustness is best addressed during training rather than through the architecture.

© 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

Keywords: Fault localization; Spatio-temporal graph neural networks; Sensor dropout robustness; Missing measurements; Dropout-aware training; Distribution grids

1. Introduction

Rapid and accurate fault localization is essential for automated restoration in modern power distribution grids. As feeders absorb growing shares of distributed energy resources and adopt self-healing schemes, how precisely a system identifies where a fault occurred, not merely that one occurred, determines how much of the network can be

* Corresponding author: M Anbazhagan
E-mail address: m_anbazhagan@cb.amrita.edu (M Anbazhagan).

1877-0509 © 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

re-energized before crews are dispatched. Traditional impedance-based and traveling-wave methods assume densely deployed measurement devices, which does not hold when micro-phasor measurement units (μPMUs) are installed at only a few locations because they are expensive and need communication infrastructure. A feeder with hundreds of buses may carry only a handful of μPMUs, so any fault-localization method must work under partial observability.

Graph neural networks (GNNs) are a natural fit: the feeder's known topology can be encoded so that message passing shares information between connected buses, letting the model infer faults even at buses carrying no sensor. Building on this, [1] introduced a spatio-temporal GNN (STGNN) coupling a GRU temporal encoder with a spatial GNN (RGCN, RGSAGE, or RGATv2) over a *measured-only* graph in which only instrumented buses are nodes, predicting one of 26 classes (25 fault locations plus no-fault). We adopt their framework and evaluation methodology as the basis for our robustness study.

That result, however, rests on an assumption that does not hold in deployed systems: that every μPMU is always available and functioning. Real sensors drop out through communication loss and hardware failure, and a model trained and evaluated only with a complete sensor set carries no guarantee of degrading gracefully when some inputs go dark. Prior work on incomplete measurement addresses a different regime, assuming a sensor-availability pattern that is fixed and known at training time (Section 2), which an inference-time failure is not. To the best of our knowledge, no previous work has studied how STGNN-based fault localization is affected when sensors fail during inference.

We therefore re-implement the three spatial STGNN encoders of [1] (RGCN, RGSAGE, RGATv2), evaluating RGSAGE in both its mean- and max-pooling forms to give four spatial baselines, together with their temporal-only GRU reference, train them on our own regenerated dataset under a common protocol whose full-observability accuracy is our reference point (Section 3.3), and make three contributions. First, we quantify how far fault-localization accuracy degrades when $k \in \{0, 1, 2, 5, 10\}$ μPMUs are dropped at inference, across all five architectures. Second, we introduce dropout-aware training, an architecture-agnostic *training method* that randomly masks measurement nodes during training, and show it recovers most of the accuracy lost to sensor dropout. Third, we report a negative result. As an *architectural* counterpart we introduce *SDR-STGNN* (Sensor-Dropout-Robust STGNN), an availability-gated network built on RGSAGE-Max that structurally silences dropped μPMUs; it matches but does not surpass dropout-aware training while costing 18% more inference latency (Section 3.6). All comparisons use identical data, splits, and evaluation protocol.

2. Related Work

Early approaches estimate the fault position from circuit physics. Krishnathevar and Ngu [2] give a generalized one-end impedance formulation that resolves the multiple-estimation ambiguity, in which a single measured impedance maps to several candidate positions on a branching feeder; Liu et al. [3] instead time the fault-induced wavefront, sharpening the arrival instants with an improved ensemble empirical mode decomposition and a Teager energy operator. Data-driven alternatives relax the need for an exact circuit model: Livani et al. [4] identify the faulted line of the IEEE 123-bus feeder with support-vector machines over post-fault RMS voltages and fault-current-indicator signals. All three assume measurement coverage dense enough to observe the relevant impedances or wavefronts, and none represents which buses are electrically adjacent. Table 1 positions the closest prior work along the axes that matter here: multi-architecture comparison, sparse observability, evaluation under test-time sensor failure, and mitigation of the resulting loss.

Encoding the feeder as a graph restores the structure that feature-vector models discard. Chen et al. [5] apply a spectral graph convolutional network with Chebyshev filters to the IEEE 123- and 37-bus systems and harden it with a data augmentation against measurement noise and data loss, the closest precedent for the training-time mitigation we study, although it is applied to a static, single-snapshot classifier over the full grid graph. Fan et al. [6] pursue reconstruction instead, pairing a variational graph autoencoder with GraphSAGE to recover latent node information under noise. Both classify a single snapshot, so neither represents how a fault signature evolves over the milliseconds following inception. Spatio-temporal models supply that time axis by placing a temporal encoder ahead of message passing, as in Nguyen et al. [7] and Ngo et al. [8], who combine a one-dimensional CNN with a GCN and with graph attention respectively. The same template recurs outside power systems, where Sachin et al. [9] cast spatio-temporal forecasting as an adaptive encoder–decoder GNN, so the design questions raised below are not peculiar to

Table 1. Positioning of the closest prior work (✓ yes, – not studied, ~ partial). "Node loss @ test" denotes whole measurement nodes going dark at inference.

Work	Approach / model	Multi-arch.	Sparse obs.	Node loss @ test	Loss mitigation
Chen et al. (2020) [5]	Spectral GCN (Chebyshev), static	–	~	~	✓
Zhang et al. (2025) [11]	GDIA-GCL: imputation + contrastive	–	✓	~	✓
Karabulut et al. (2026) [1]	STGNN: RGCN, RGSAGE, RGATv2	✓	✓	–	–
This work	**STGNN + node drop-out study**	✓	✓	✓	✓

feeder data. This paper builds directly on Karabulut et al. [1], who benchmark RGCN, RGSAGE, and RGATv2 spatial encoders on a shared GRU temporal encoder over the IEEE 123-bus feeder. Their *measured-only* graph promotes only instrumented buses to nodes, so message passing operates over the sensing infrastructure rather than the physical feeder, beating a purely recurrent baseline by up to 11 F1 points at roughly one-sixth of the training time. A companion study [10] instead varies the grid configuration the detector is transferred to.

Zhang et al. [11] confront incomplete measurement directly: a training-free graph Dirichlet imputation reconstructs absent bus measurements by minimizing the graph Dirichlet energy, and a GCN trained with graph contrastive learning then keeps fault signatures separable under the resulting scarcity. This is the most developed treatment of missing data in this task, and it is instructive for what it assumes: that the set of unmeasured buses is a fixed property of the network, known when both the imputer and the classifier are built.

Beyond this task, the same graph formulation carries across domains that share nothing but relational structure. Jayapal and Annamalai [12] build a graph over DeepLab-segmented lymph-node regions and classify metastasis with a GNN applied to a Hessian-based locally linear embedding of the node features, and Asiri and Somasundaram [13] identify illicit Bitcoin transactions with a graph convolutional network over the transaction graph; both report gains over the non-graph baselines they compare against. Closest to our concern is traffic forecasting, whose sensor streams fail the way μPMUs do: Kirthan et al. [14] pair diffusion graph convolutions and dilated temporal convolutions with Monte Carlo dropout to separate aleatoric from epistemic uncertainty, and report that the resulting model holds up better than ST-GCN, DCRNN, and Graph WaveNet under sensor dropout. That result establishes the concern in a regression setting with dense sensing, where a lost detector chiefly degrades the forecast at its own node. Fault localization differs on both counts: the quantity to be predicted is a discrete location that is usually not instrumented at all (Section 3.1), so a lost μPMU withdraws evidence about buses other than its own, and the damage cannot be read off any single node's error.

These lines of work leave the two right-hand columns of Table 1 almost empty: no prior fault-localization method combines sparse observability with an evaluation of sensors that fail *after* deployment and a mitigation for the accuracy lost when they do. Yet a μPMU depends on communication infrastructure and periodic maintenance, so on a feeder instrumented with 25 of them, losing a handful is an ordinary operating event. Our objective follows: to measure how far STGNN localization falls as μPMUs are lost at inference, across architectures rather than for a single model, and to separate what can be fixed by training from what requires changing the model itself.

3. Materials and Methods

3.1. Grid Simulation and Dataset

We generate our own corpus on the IEEE 123-bus distribution feeder, a standard radial benchmark, simulated in OpenDSS through the OpenDSSDirect.py interface. Simulations run in daily (quasi-static time-series) mode at a 1 ms step, so the loads evolve along a predefined profile and each recorded sample is a converged power-flow solution rather than a single operating-point snapshot. This is an RMS, phasor-domain representation: it captures how the voltage envelope evolves across fault inception and over the window that follows, but not the electromagnetic transient itself (traveling waves, DC offset, sub-cycle harmonics), which a quasi-static solver does not model. The fault signature our models learn is therefore the post-inception RMS depression and its spatial pattern across the feeder, and our accuracies should be read as applying to that class of signal. Table 2 gives the full simulation configuration, feature layout, and per-split window counts, every entry computed directly from the generated dataset.

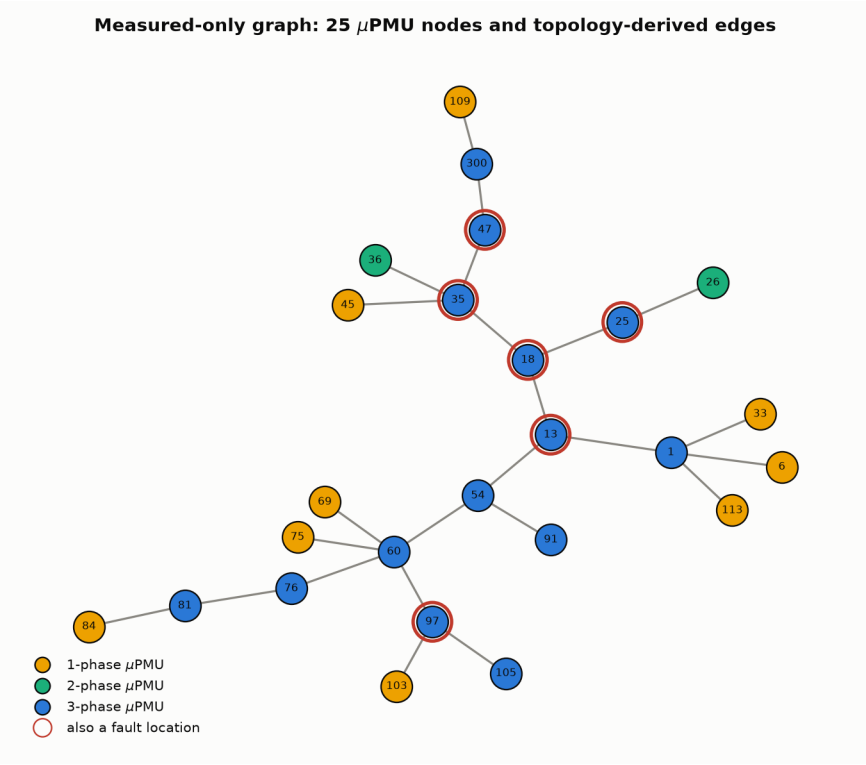


Fig. 1. The measured-only graph of 25 μPMU buses (colored by phase count) and 24 topology-derived edges, with red rings on the 6 buses that are also fault locations.

Partial observability is modeled by instrumenting only 25 buses with μPMUs, following the placement of [1] (single-, two-, and three-phase units reporting three-phase RMS voltages). A reduced *measured-only* graph is built from these buses (Fig. 1): the 25 μPMUs are its nodes and their connectivity is inherited from the feeder topology, giving a connected tree with 24 edges, so message passing operates on the sensing infrastructure rather than the complete physical feeder. Only 6 of the 25 candidate fault locations coincide with an instrumented bus; the remaining 19 are never directly observed, the central difficulty the spatial model must overcome.

Two encoding choices behind Table 2 carry methodological weight. First, the availability mask defines what "missing" means throughout this paper: $m_p = 1$ marks an available measurement and $m_p = 0$ a missing phase or a dropped sensor, so a μPMU that fails is encoded exactly like one that was never installed. Second, the 70/15/15 partition is taken at the *event* level, so the 40 sliding windows of one event never span more than one split and no window of a test event is seen during training. Voltage channels are z-score normalized using training-split statistics only. We adopt the base paper's dataset-generation methodology but build our own corpus (Section 3.3); since every model is trained and evaluated on it with identical splits, the comparisons remain fair. Fault severity varies sharply with fault resistance: for a three-phase-to-ground fault at bus 101, the deepest-sagging μPMU channel drops 0.83 p.u. below its pre-fault level at 0.1 Ω, 0.22 p.u. at 1 Ω, and only 0.01 p.u. at 10 Ω, which is why high-impedance faults are the hardest class to localize throughout this study.

3.2. Baseline Architectures: GRU, RGCN, RGSAGE, RGATv2

We evaluate the three spatial STGNN encoders of [1] (RGCN, RGSAGE, and RGATv2), taking RGSAGE in both its mean- and max-pooling forms so that four spatial baselines result, together with the temporal-only GRU that serves as their non-spatial reference; the GRU has no message-passing stage and is therefore not itself a spatio-temporal GNN. All five follow the designs of [1] unchanged, the one exception being spatial depth, a configuration choice discussed below. On top of the strongest baseline, RGSAGE-Max, this work adds a training method, *dropout-aware*

Table 2. Dataset configuration and event-level 70/15/15 split of the 50-run corpus.

Simulation configuration	
Test feeder / simulator	IEEE 123-bus, 4.16 kV, 60 Hz; OpenDSS (OpenDSSDirect.py), Daily mode, 1 ms step
Fault buses / μPMUs	25 / 25 (single-, two-, and three-phase units)
Fault types / resistances	11 (3 LG, 3 LL, 3 LLG, 2 three-phase) / 3 (0.1, 1, 10 Ω)
Load multiplier	$\mathcal{U}(0.5,\ 1.3)$ per run
Scenarios × runs	$25 \times 11 \times 3 = 825$ scenarios × 50 runs
Record / windowing	60 ms record (40 ms pre-fault); 20-step (20 ms) windows, 40 per event
Node features F / classes	6 ($[V_1, V_2, V_3]$ + availability mask $[m_1, m_2, m_3]$) / 26 (25 locations + no-fault)

Window-level distribution (event-level split)	
Split	Fault / No-fault / Total windows (events)
Train	577,500 / 577,500 / 1,155,000 (28,875)
Validation	124,000 / 124,000 / 248,000 (6,200)
Test	123,500 / 123,500 / 247,000 (6,175)
Total	825,000 / 825,000 / 1,650,000 (41,250)

training (Section 3.5), and an architecture, *SDR-STGNN*, evaluated as an ablation against it (Section 3.6). All five share a common template: a temporal encoder produces one embedding per μPMU node, an optional spatial stage exchanges information over the measured-only graph, and a shared per-node head with soft voting yields the graph-level prediction. Let $x_v \in \mathbb{R}^{S\times F}$ be the input at μPMU node v of the $N = 25$ node measured-only graph, over the $S = 20$ step window with $F = 6$ channels, and let $\mathcal{N}(j)$ denote the neighbors of node j in that graph. A single-layer GRU with 128 hidden units, shared across nodes and applied to each node independently, gives the node embedding of Eq. (1):

$$h_v^{(0)} = \mathrm{GRU}(x_v) \in \mathbb{R}^{128}, \qquad v = 1, \ldots, N. \tag{1}$$

The GRU-only baseline stops at Eq. (1) and cannot exploit spatial relationships. The four GNN variants stack $L = 4$ message-passing layers (hidden dimension 64, ReLU between layers, self-loops added) over that graph and differ only in the aggregation operator: *RGCN* propagates over the symmetrically normalized adjacency (Eq. (2)); *RGSAGE-Mean* and *RGSAGE-Max* concatenate a node's own embedding with a mean- or max-pooled neighborhood before projecting (Eq. (3)) [15]; and *RGATv2* applies $H = 4$ head GATv2 attention over each node's in-neighbors (Eq. (4)) [16]:

$$h^{(l+1)} = \mathrm{ReLU}\Big(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}\, h^{(l)} W^{(l)}\Big), \tag{2}$$

$$h_j^{(l+1)} = \mathrm{ReLU}\Big(W^{(l)}\Big[h_j^{(l)} \,\Big\|\, \rho(\{\, h_i^{(l)} : i \in \bar{\mathcal{N}}(j)\,\})\Big]\Big), \tag{3}$$

$$h_j^{(l+1)} = \mathrm{ReLU}\left(W_{\mathrm{o}}^{(l)}\Big\|_{c=1}^{H} \sum_{i\in\bar{\mathcal{N}}(j)} \alpha_{ji}^{(l,c)}\, W^{(l,c)} h_i^{(l)}\right), \tag{4}$$

where $\tilde{A} = A + I$ is the adjacency with self-loops and $\tilde{D}$ its degree matrix, $\bar{\mathcal{N}}(j) = \mathcal{N}(j) \cup \{j\}$ is the closed neighborhood (self-loops are added in every variant), $W^{(l)}$ are the layer weights, $W_{\mathrm{o}}^{(l)}$ projects the H concatenated attention heads back to the layer width, $\|$ is concatenation, ρ is mean or max pooling, and $\alpha_{ji}^{(l,c)}$ is the GATv2 attention coefficient of head c. The ReLU separates consecutive hops and is omitted after the last one, whose output passes directly to the classifier head. These operators are adopted unchanged from [1].

Spatial depth (configuration choice): The base paper does not state how many message-passing layers its GNN variants use, so the depth had to be selected here. A single hop gave substantially weaker localization, because most fault locations lie more than one hop from any μPMU (only 6 of 25 fault buses are instrumented); we therefore stack *four* message-passing layers in every GNN variant, fixed for all experiments (Section 4).

Classification and aggregation: A shared head (batch normalization, ReLU, dropout $p = 0.35$, linear projection to 26 classes) maps the final embedding of every node to logits z_v. At inference the per-node class probabilities are summed over all N nodes and the arg-max is the graph-level prediction (soft voting, Eq. (5)). Training minimizes the unweighted node-level cross-entropy of Eq. (6), in which each window's event label y supervises all nodes; class balance is handled by the balanced corpus (Section 3.7) rather than by loss weighting.

$$\hat{y} = \arg \max_{c \in \{0,\ldots,25\}} \sum_{v=1}^{N} \mathrm{softmax}(z_v)_c, \tag{5}$$

$$\mathcal{L} = \frac{1}{N} \sum_{v=1}^{N} \mathrm{CE}(z_v,\, y)\,. \tag{6}$$

3.3. Deviations from the Base Setup

Our setup follows [1] (feeder, μPMU placement, fault taxonomy, windowing, soft-voting evaluation, and an unweighted cross-entropy objective) but differs in four documented respects: a spatial depth of four message-passing hops, which the base paper leaves unspecified; a 1.65 M-window corpus against its 2.5 M; a longer schedule of 30 epochs with gradient clipping and learning-rate warmup, against its 15 epochs for the GNN models and 11 for the GRU; and omission of the reconfigured ("green") topology. All comparisons here use our own corpus and splits, so they are internally consistent and should be read against our own 0.85 reference point rather than the base paper's absolute scale.

3.4. Sensor-Dropout Evaluation Protocol

We evaluate robustness to μPMU failures entirely at *inference* time, with all model parameters fixed, which isolates robustness from any training-time mitigation.

Missingness model: To simulate a lost sensor we select k of the 25 μPMUs and, for every test window, zero all six input channels of each selected node, both the RMS-voltage and the availability-mask channels. Zeroing the mask is essential: a zero voltage can be a valid reading, so the mask is the definitive indicator that a node is unobserved. This reproduces the dataset's encoding of an absent sensor (Section 3.1).

Graph handling: A dropped node keeps its edges and still participates in message passing, but its temporal encoder receives an all-zero sequence and emits a zero-valued embedding that propagates to neighbors, exactly the non-informative message the availability-gated model (Section 3.6) is designed to suppress. All models see the same graph, so differences arise solely from the message-passing mechanism. (An edge-removal policy was implemented but not used, to avoid confounding topology with dropout.)

Dropout sweep and metrics: We sweep $k \in \{0, 1, 2, 5, 10\}$ (0/4/8/20/40% loss), with $k = 0$ the full-observability reference. Dropped sensors are sampled uniformly at random per window, driven by fixed seeds reused across all models so that every architecture sees an identical sequence of dropout realizations on the same frozen test split (Table 2). Each (model, k) point is averaged over $n_{\text{seed}} = 3$ mask realizations and, where stated, over training seeds 42/43/44. Performance is the soft-voted macro-F1 over the 26 classes, giving a macro-F1-versus-k *degradation curve*, from which we derive the *retention* $R(k)$ of Eq. (7) and the *normalized area under the retention curve* (AUDC) of Eq. (8):

Algorithm 1 Dropout-aware training; only the per-batch masking of lines 4–5 differs from standard training.

Require: training batches $\mathcal{B}$; dropout levels $\mathcal{K} = \{0, 1, 2, 5\}$; epochs E; model f_θ
Ensure: dropout-aware weights θ
1: **for** $e = 1$ **to** E **do**
2: reseed the batch-masking RNG ▷ fresh dropout each epoch
3: **for** each mini-batch $(x, y) \in \mathcal{B}$, $x \in \mathbb{R}^{B\times N\times S\times 6}$ **do**
4: $k \leftarrow \text{Uniform}(\mathcal{K})$ ▷ one level per batch
5: for every window $b \le B$, draw k nodes uniformly and zero all six channels of x_b at those nodes
6: $Z \leftarrow f_\theta(x)$; $\mathcal{L} \leftarrow$ mean of Eq. (6) over the batch ▷ $Z \in \mathbb{R}^{B\times N\times 26}$
7: backpropagate; clip gradient norm to 1.0; AdamW step ▷ LR warmup over first 3 epochs
8: **end for**
9: **end for**

$$R(k) = \frac{\text{F1}(k)}{\text{F1}(0)}, \tag{7}$$

$$\text{AUDC} = \frac{1}{k_{\max} - k_{\min}} \int_{k_{\min}}^{k_{\max}} R(k)\, dk \in [0, 1]. \tag{8}$$

Here $k_{\min} = 0$ and $k_{\max} = 10$ are the extremes of the sweep, and Eq. (8) is computed by trapezoidal integration over $k \in \{0, 1, 2, 5, 10\}$; a higher AUDC means slower degradation. Both metrics are relative to a model's own clean score, so they compare how gracefully models degrade, not how accurate they are.

3.5. Dropout-Aware Training

Dropout-aware training removes the train-test mismatch exposed in Section 3.4: the baselines train with all 25 μPMUs available but are evaluated under partial observability. We apply the evaluation missingness model as training-time augmentation, drawing a dropout level $k \sim \text{Uniform}\{0, 1, 2, 5\}$ per batch and zeroing k randomly chosen nodes before each forward pass (Algorithm 1). Because k varies batch to batch, every epoch mixes full- and partial-observability cases, so no μPMU is consistently available and the network cannot depend on any individual sensor. The augmentation touches only the input features, not the graph, architecture, or optimizer, so it applies unchanged to all baselines and to SDR-STGNN (Section 3.6). Masking graph elements during training is itself a known regularizer (e.g., DropEdge [17]); our contribution is not the primitive but framing μPMU dropout as a train-test mismatch and studying its mitigation.

3.6. SDR-STGNN: Availability-Gated Graph Aggregation

SDR-STGNN (Sensor Dropout-Robust Spatio-Temporal Graph Neural Network) addresses sensor dropout architecturally and is included as an ablation against dropout-aware training. All five baselines receive the availability mask as input channels $[m_1, m_2, m_3]$, yet none of the operators in Eqs. (2)–(4) uses that information, so a dropped node keeps contributing its zero-valued embedding as a non-informative message. SDR-STGNN makes the aggregation of the strongest baseline, RGSAGE-Max, *availability-gated*, so dropped sensors send no messages at all.

Availability: For each input window we derive a binary availability indicator for every node from its three availability-mask channels, given by Eq. (9):

$$a_i = \mathbf{1}\left\{ \max_{s} \max_{p\in\{1,2,3\}} m_{i,p}(s) > 0 \right\} \in \{0, 1\}, \tag{9}$$

where $m_{i,p}(s)$ is the mask value of phase p at node i and step s; thus $a_i = 1$ iff any mask channel is active at any step in the window. Every node of the measured-only graph carries a μPMU, so a single- or two-phase unit still yields $a_i = 1$; under the protocol of Section 3.4 the nodes with $a_i = 0$ are exactly those whose sensor has been dropped.

Gated aggregation: Eq. (10) replaces the RGSAGE update of Eq. (3) at each of the four message-passing layers, writing h_i for $h_i^{(l)}$ and h'_j for $h_j^{(l+1)}$: the max-aggregation is restricted to available source nodes, and the self-representation is gated by the same availability indicator:

$$h'_j = \mathrm{ReLU}\Big(W\left[a_j h_j \,\Big\|\, \mathrm{agg}_j\right]\Big), \qquad \mathrm{agg}_j = \max_{\substack{i\in\bar{N}(j)\\ a_i=1}} h_i, \tag{10}$$

where W is the layer projection of Eq. (3) and $\bar{N}(j)$, $\|$, and the inter-hop ReLU convention are as defined there. Nodes with $a_i = 0$ send no messages, and $\mathrm{agg}_j = \mathbf{0}$ when the entire closed neighborhood of j is unavailable. The availability indicators are computed once from the input window (Eq. (9)) and reused across all four layers. The gate adds no parameters, and the shared GRU encoder, four-hop depth, classifier head, and soft voting are identical to RGSAGE-Max; when every sensor is available ($a_i = 1$ for all i), Eq. (10) reduces exactly to the standard RGSAGE-Max update, so the two are identical under full observability ($k = 0$). SDR-STGNN is trained with the dropout-aware augmentation of Section 3.5, so the gates are exercised throughout training. Because everything else is held identical, comparing it against the ungated dropout-aware model separates the training-time augmentation from the structural gate (Section 4).

3.7. Experimental Setup

All models share one training and evaluation configuration, listed in Table 3. Apart from the variable under study (the message-passing architecture or the augmentation of Section 3.5), every part of the pipeline is fixed, so any difference is attributable to that variable alone. The points not covered by Table 3 are:

- *Batching:* an event-chunk sampler keeps all sliding windows of an event together, consistent with the event-level partition of Table 2.
- *Objective:* imbalance is handled by dataset construction rather than loss weighting. The corpus is 50% no-fault and the $25 \times 11 \times 3$ factorial design represents the 25 fault locations uniformly, so inverse-frequency weighting is unnecessary, matching the unweighted objective of [1].
- *Determinism:* parameter initialization, mini-batch ordering, and the dropout-aware masking are all controlled by one integer seed; no early stopping is used beyond best-validation checkpointing.

4. Results

We first establish the baselines under full observability, then quantify how those fixed models degrade as μPMUs are removed at inference (the paper's primary evaluation), then compare dropout-aware training against the availability-gated SDR-STGNN, and finally test whether the training-time fix generalizes across architectures. Unless otherwise stated, results are computed on the frozen test split of Table 2 and reported as mean ± standard deviation over training seeds 42, 43, and 44, consistent with Section 3.7.

4.1. Baseline Accuracy Under Full Observability

Table 4 summarizes the five baselines under full observability ($k = 0$); the two macro-F1 columns carry ± one standard deviation over seeds 42/43/44, while the per-resistance columns are seed means. The four spatio-temporal GNN models consistently outperform the temporal-only GRU, a gap that shows the importance of spatial message passing here: with only 6 of the 25 fault locations instrumented (Section 3.1), localization usually requires information to propagate to unmeasured buses, for which Eq. (1) alone provides no mechanism. For every model the graph-level

Table 3. Training, evaluation, and software configuration shared by all models.

Model configuration	
Temporal encoder	1-layer GRU, 128 hidden units, shared across nodes
Spatial layers	4 message-passing layers, hidden dimension 64
Classifier head	BatchNorm → ReLU → dropout 0.35 → linear (26); attention dropout 0.3 (RGATv2)
Graph-level readout	Soft voting over the 25 node predictions
Optimization	
Optimizer	AdamW, learning rate 10^{-3}, weight decay 10^{-4}; batch size 32, 30 epochs
Schedule / clipping	Linear LR warmup over first 3 epochs, constant thereafter; global norm 1.0
Loss / model selection	Node-level unweighted cross-entropy; best validation graph macro-F1
Evaluation and reporting	
Training seeds	42, 43, 44 (mean ± std reported)
Dropout levels k	0, 1, 2, 5, 10 of 25 μPMUs (0–40% loss)
Mask realizations	3 per training seed, seeds shared across models
Metrics	Soft-voted macro-F1, retention, AUDC
Hardware and software	
GPU	1 × NVIDIA T1000 (8 GB)
Software stack	Python 3.12, PyTorch 2.5.1 (CUDA 12.1), PyTorch Geometric 2.8, OpenDSSDirect.py 0.9.4, NumPy 2.5, scikit-learn 1.9

Table 4. Baseline macro-F1 under full observability, as mean±std over seeds 42/43/44, with the best value per column in bold.

Model	Graph-level	Node-level	Macro-F1 by fault resistance		
	Macro-F1	Macro-F1	0.1 Ω	1 Ω	10 Ω
RGSAGE-Max	**0.846 ± 0.007**	**0.718 ± 0.008**	**0.949**	**0.893**	**0.676**
RGSAGE-Mean	0.786 ± 0.023	0.660 ± 0.025	0.906	0.843	0.587
RGCN	0.712 ± 0.017	0.601 ± 0.019	0.874	0.792	0.428
RGATv2	0.522 ± 0.087	0.460 ± 0.080	0.630	0.574	0.312
GRU	0.286 ± 0.009	0.150 ± 0.003	0.410	0.292	0.105

macro-F1 exceeds the node-level macro-F1, so the soft vote of Eq. (5) consistently improves on individual node predictions.

Accuracy decreases monotonically as fault resistance increases, for every model: RGSAGE-Max, for example, drops from 0.949 at 0.1 Ω to 0.676 at 10 Ω, reflecting the weaker voltage signatures of high-impedance faults (Section 3.1). Fault detection, in contrast, is effectively saturated: all models achieve a no-fault recall of at least 0.9999 and a no-fault precision of at least 0.972, so the primary challenge is localization rather than detection. Unlike [1], RGATv2 is the weakest GNN model here (0.522 ± 0.087) and the highest in variance across seeds; that spread reflects training instability at the adopted four-layer depth. Stabilizing RGATv2 at that depth is left as future work.

These results also justify the spatial depth of four: a single-seed diagnostic depth sweep on RGSAGE-Max rises monotonically from 0.509 (1 hop) to 0.806 (4 hops, reduced budget), with the full 30-epoch recipe reaching 0.846 and no sign of over-smoothing, confirming the enlarged receptive field as the primary driver of accuracy.

4.2. Accuracy Degradation Under Sensor Dropout

We now freeze each baseline and evaluate it under the dropout sweep of Section 3.4. Table 5 summarizes the degradation curves through AUDC (Eq. (8)) and retention at the most severe level ($k = 10$, 40% sensor loss); Fig. 2 shows the full curves. Every model declines as sensors are removed, and no baseline retains more than a third of its full-observability macro-F1 at $k = 10$.

The results reveal a clear trade-off between clean accuracy and robustness; the training-seed standard deviation stays below 0.06 at every k for four of the five models and reaches 0.09 for RGATv2. RGSAGE-Max, the strongest model under full observability, is the least robust, and the inversion is not an artifact of normalizing by each model's

Table 5. Degradation under sensor dropout for standard (Std) and dropout-aware (DA) training, as mean ± std over seeds 42/43/44 with three mask realizations each. Bold marks the best *absolute* macro-F1 per column; relative columns are unbolded, normalizing by each model's own clean score.

Model	Standard-training macro-F1 at k					DA macro-F1		AUDC		Ret.@10 (%)	
	k=0	k=1	k=2	k=5	k=10	k=0	k=10	Std	DA	Std	DA
RGATv2	0.522	0.490	0.444	0.302	0.157	0.553	0.505	0.626±0.112	0.961±0.014	31.0±16.4	91.4±2.8
RGCN	0.712	0.664	0.610	**0.418**	**0.164**	0.613	0.533	0.608±0.047	0.940±0.011	23.2± 7.0	87.0±1.9
RGSAGE-Mean	0.786	0.720	**0.647**	0.393	0.108	0.673	0.550	0.541±0.041	0.922±0.034	13.8± 6.4	81.9±8.4
GRU	0.286	0.273	0.252	0.111	0.051	0.300	0.078	0.522±0.081	0.781±0.038	17.9± 4.4	25.4±5.8
RGSAGE-Max	**0.846**	**0.736**	0.590	0.236	0.048	**0.839**	**0.674**	0.402±0.020	0.913±0.006	5.7± 1.8	80.3±1.3

own clean score: the curves cross in *absolute* macro-F1 as well. RGATv2's nominal top-robustness position is indicative only, resting on the lowest clean accuracy of the GNNs and a retention spread that overlaps RGCN's, but the inversion does not depend on it: excluding RGATv2 entirely, RGSAGE-Max still retains 5.7% against RGCN's 23.2%. Clean accuracy therefore does not predict robustness to sensor loss, since the most accurate model degrades the fastest. This is consistent with how the aggregation operators treat the zeroed embedding a dropped node propagates (Section 5), and it motivates the two mitigations of Sections 3.5 and 3.6.

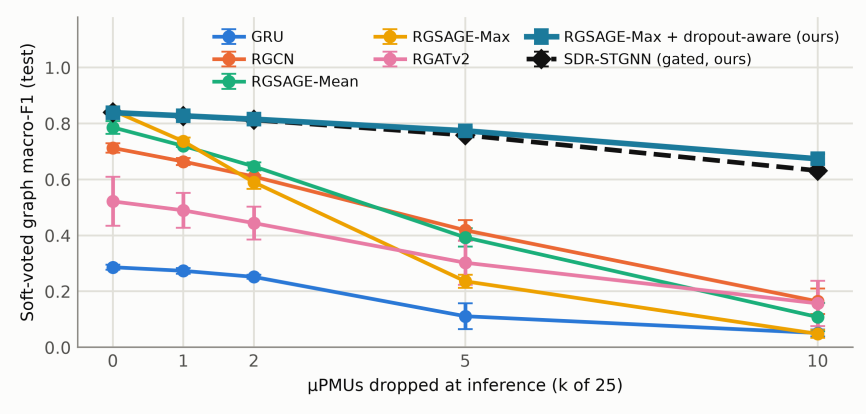


Fig. 2. Soft-voted macro-F1 against the number k of dropped μPMUs, for the five standard-trained baselines and both RGSAGE-Max mitigations: dropout-aware training (blue squares) and the availability-gated SDR-STGNN (black dashed), itself dropout-aware trained. The two nearly coincide — the gate adds no robustness over the augmentation. Error bars are ±1 std over seeds 42/43/44.

4.3. Effect of Dropout-Aware Training

Table 6 compares four ways to cope with sensor loss on the identical sweep, all on the RGSAGE-Max backbone: standard training (dropped nodes zeroed), an impute-then-infer baseline, dropout-aware training, and the availability-gated SDR-STGNN; the dropout-aware and SDR-STGNN curves are also plotted in Fig. 2.

Imputation baseline: A natural alternative to a training-time fix is to reconstruct dropped sensors before inference. As a lightweight control, the no-training analogue of graph-based imputation [11], we fill each dropped node's three voltage channels with the per-timestep mean of its *available* undirected 1-hop neighbors and present the result to the standard-trained model as observed (mask set to 1, the most favorable setting for imputation); dropped nodes are selected exactly as in the zeroing sweep, so the only change is the fill. This imputer gains little over zeroing (AUDC 0.402 → 0.476; $k = 10$ retention 5.7% → 11.8%, a difference that is not significant at three seeds, Section 4.5) and stays far below dropout-aware training (AUDC 0.913), because the reconstructed values are still fed to a model never trained to distrust them. Unlike the learned pipeline of [11], it needs no advance knowledge of which sensors will fail — exactly what an inference-time failure withholds.

Dropout-aware training raises the AUDC of RGSAGE-Max from 0.402 to 0.913 and its $k = 10$ retention from 5.7% to 80.3%: where the standard model collapses to a macro-F1 of 0.048 at 40% sensor loss, the dropout-aware variant

Table 6. Test-time and training-time mitigations on the RGSAGE-Max backbone, as mean ± std over seeds 42/43/44, best per column in bold. All four share one backbone and one clean score, so the relative columns are directly comparable here, unlike those of Table 5. Imputation is a no-op at k=0.

Variant	k=0	k=1	k=2	k=5	k=10	AUDC	Ret.@10 (%)
Standard training (zeroing)	**0.846±0.007**	0.736±0.013	0.590±0.024	0.236±0.023	0.048±0.015	0.402±0.020	5.7±1.8
Mean-imputation (standard)	0.846±0.007	0.739±0.017	0.637±0.028	0.336±0.043	0.099±0.033	0.476±0.042	11.8±4.0
Dropout-aware training	0.839±0.011	**0.827±0.011**	**0.815±0.011**	**0.774±0.013**	**0.674±0.017**	**0.913±0.006**	**80.3±1.3**
SDR-STGNN (gated)	0.839±0.002	0.827±0.003	0.813±0.004	0.758±0.006	0.631±0.010	0.892±0.007	75.2±1.3

still reaches 0.674 ± 0.017, an absolute gain of 0.626 macro-F1 points. Clean performance is essentially unchanged (0.839 ± 0.011 against 0.846 ± 0.007 at $k = 0$, within one standard deviation and not significant at $p = 0.181$; Section 4.5), and because the augmentation alters only the training data it adds no parameters and no measurable inference-time cost: measured on one NVIDIA T1000, the standard and dropout-aware checkpoints have identical parameter counts (95,258) and single-event latency (0.639 against 0.640 ms), the augmentation's only overhead being 2.7% on training time per epoch.

Targeted (worst-case) dropout: The sweep above removes μPMUs uniformly at random and therefore reports average-case loss. To bound the worst case we repeat it dropping instead the k μPMUs *nearest the fault*, ranked by hop distance over the full 123-bus topology rather than the measured-only graph, on which a fault bus is generally not a node; selection is then a deterministic function of the fault location, so all models see identical masks without a shared seed, and the no-fault half of the corpus, which has no fault bus, retains random selection. Targeting is markedly more damaging in the regime where the model is still usable: standard RGSAGE-Max falls to 0.448 ± 0.039 at $k = 1$ against 0.736 ± 0.013 under random loss ($p = 0.010$), a 3.6× larger drop from its clean score, and to 0.361 against 0.590 at $k = 2$ ($p = 0.017$); from $k \geq 5$ the two policies agree, the standard model having collapsed under either. Dropout-aware training is penalized by targeting as well (AUDC 0.913 → 0.747, $p = 0.003$; $p \leq 0.014$ at every individual k), so worst-case loss is a distinct stressor rather than one the mitigation absorbs. It nonetheless retains far more than the standard model under the same targeting (+0.307 to +0.473 macro-F1, $p \leq 0.008$), and from $k = 2$ upward its *worst-case* score exceeds the standard model's *average-case* score (0.695 against 0.590 at $k = 2$, $p = 0.042$; 0.521 against 0.048 at $k = 10$, $p = 0.004$), though at $k = 1$ the two are indistinguishable (+0.018, $p = 0.33$). The training-time mitigation therefore transfers to a failure mode it was never trained on.

Persistent (per-event) dropout: Both sweeps above redraw the dropped set for every window, whereas a failed μPMU stays dark for the whole event. We therefore repeat the random sweep drawing the dropped set once per event and holding it across all 40 of that event's windows, with selection, zeroing, and the k grid otherwise unchanged, so persistence is the only variable. It makes no difference: no point on either curve moves by more than 0.003 macro-F1 (standard AUDC 0.402 → 0.401, dropout-aware 0.913 → 0.913; $k = 10$ retention 5.7% → 5.6% and 80.3% → 80.1%), and across the twelve paired comparisons no difference survives Holm correction (min $p_{\text{Holm}} = 0.46$). The mitigation claim is likewise unchanged under the coherent model (AUDC +0.512, $k = 10$ retention +74.5 points, both $p \leq 0.0006$). The reason is that our decision rule is per window: soft voting pools the 25 nodes of a single window (Eq. (5)), never the 40 windows of an event, so a window's prediction depends only on its own mask, whose marginal distribution is identical under both schemes. The equivalence should therefore not be assumed for an *event-level* rule, which per-window resampling would flatter by supplying 40 differently-masked views of a failure that in reality presents only one.

The availability-gated SDR-STGNN matches that robustness but does not exceed it: its AUDC is 0.892 and its $k = 10$ retention 75.2%, both slightly below the ungated model, and its curve lies below the dropout-aware curve for all $k \geq 1$. At $k = 0$ the gate of Eq. (10) is inactive, so the architecture reduces exactly to its backbone and the two dropout-aware variants score identically (0.839, within seed noise). The gate is also not free: recomputing availability at every hop raises single-event latency by 18% (0.755 against 0.639 ms) for no robustness gain. The structural gate thus adds nothing beyond the training-time augmentation, leaving dropout-aware training the most effective and most practical mitigation evaluated here.

4.4. Dropout-Aware Training Across Architectures

To test whether the gain is specific to the RGSAGE-Max backbone, we retrain every baseline with the identical augmentation and re-evaluate under the same sweep; the resulting values appear in the dropout-aware columns of Table 5. Robustness improves for all five architectures. The four spatial GNNs rise from AUDC 0.40–0.63 to 0.91–0.96 and from at most 31% to 80–91% retention at $k = 10$. The cost in clean accuracy, however, is not uniform: RGSAGE-Max loses under 0.01 macro-F1 at $k = 0$, but RGCN drops 0.099 (0.712 → 0.613) and RGSAGE-Mean 0.113 (0.786 → 0.673), so the result that robustness comes at no cost to clean accuracy holds for the backbone we adopt rather than for the family as a whole; we did not isolate the cause. Because retention normalizes by each model's own clean score, the relative leaders are again not the strongest in absolute terms: dropout-aware RGSAGE-Max reaches 0.674 macro-F1 at $k = 10$, well above RGATv2's 0.505 despite RGATv2's higher retention. The GNN-free GRU also improves (AUDC 0.52 → 0.78) but its retention stays limited (25%), since without a spatial mechanism it cannot exploit surviving sensors. Robustness recovery is therefore a general property of the training scheme, strongest where the architecture can route information around a failure.

4.5. Statistical Significance

Every comparison above is a mean over three training seeds, which is a small sample, so we state explicitly which differences survive a test. Table 7 reports paired two-sided t-tests on the seed-wise differences ($n = 3$, df $= 2$) for the six mitigation claims, Holm–Bonferroni adjusted across the family with the two expected null results included rather than exempted. The seed is the paired unit because every variant is trained from the same three seeds on identical data and splits, so the pairing removes seed-to-seed variation in training difficulty. We do not rely on a Wilcoxon signed-rank test, whose smallest attainable two-sided p-value at $n = 3$ is 0.25.

The positive claims survive adjustment. Dropout-aware training gains 0.626 macro-F1 at $k = 10$ (95% CI $[0.557, 0.695]$, $p_{\text{Holm}} = 0.003$), 74.6 percentage points of retention and 0.511 AUDC over standard training (both $p_{\text{Holm}} = 0.003$), and exceeds mean-imputation by 0.574 macro-F1 at $k = 10$ ($p_{\text{Holm}} = 0.008$). Extending the same test to every level and every architecture, the gain is significant at each $k \geq 1$ ($p_{\text{Holm}} \leq 0.042$) and the AUDC gain is significant for all five architectures ($p_{\text{Holm}} \leq 0.028$), so the cross-architecture claim of Section 4.4 does not rest on the backbone alone. The two claims we make *as* null results test as null: the clean-accuracy change at $k = 0$ is -0.008 with a CI spanning zero ($p = 0.181$), and dropout-aware training's 0.021 AUDC edge over SDR-STGNN is not significant ($p = 0.098$), which is the statistical form of "matches but does not surpass". Mean-imputation's 0.051 gain over zeroing at $k = 10$ is likewise not significant ($p = 0.102$), sharpening Section 4.3: on this evidence *this* imputer — a training-free neighbor-mean fill — is not distinguishable from leaving the dropped node at zero, a statement about the one control we ran.

Two cross-architecture orderings need an explicit caveat, which reflects the limit of a three-seed design rather than a weakness in the effects themselves. The clean-accuracy ranking of Table 4 reaches only $p = 0.023$ for its top pair (RGSAGE-Max over RGSAGE-Mean) and no adjacent pair survives Holm adjustment across the four. Neither does the accuracy–robustness inversion: RGCN's absolute macro-F1 at $k = 10$ exceeds RGSAGE-Max's in every one of the three seeds, by a factor of 2.4 to 6.7 ($\Delta = 0.116$), yet $p = 0.076$, because the *size* of the gap varies with the seed even though its sign does not. A complementary bootstrap over the 6,175 test events — events, not windows, since one fault contributes 40 correlated windows — locates that noise: its interval for the top pair's clean-accuracy gap is roughly five times narrower than the same pair's seed-level interval. What bounds these orderings is therefore the number of training seeds, not the size of the test set, and we report them as consistent observations at three seeds rather than as significant differences.

5. Discussion

5.1. Robustness Ranking of the Baseline Architectures

The five baselines expose an inverse relationship between clean accuracy and robustness: the accuracy ranking (RGSAGE-Max > RGSAGE-Mean > RGCN > RGATv2 > GRU) is essentially reversed by the robustness (AUDC)

Table 7. Paired two-sided t-tests over seeds 42/43/44 ($n = 3$, df = 2) on the mitigation claims, all on the RGSAGE-Max backbone. Δ is the mean seed-wise difference (F1 points, AUDC, or percentage points for Ret.@10); p_{Holm} is Holm–Bonferroni adjusted over the six tests. Std = standard, DA = dropout-aware.

Comparison	Δ	95% CI	p	p_{Holm}
DA vs. Std, clean F1 (k=0)	−0.008	[−0.024, +0.009]	0.181	0.196
DA vs. Std, F1 at k=10	+0.626	[+0.557, +0.695]	<0.001	0.003
DA vs. Std, Ret.@10	+74.6	[+67.2, +82.0]	<0.001	0.003
DA vs. Std, AUDC	+0.511	[+0.450, +0.571]	<0.001	0.003
DA vs. imputation, F1 at k=10	+0.574	[+0.449, +0.699]	0.003	0.008
DA vs. SDR-STGNN, AUDC	+0.021	[−0.010, +0.052]	0.098	0.196

ranking (Tables 4–5). The clearest signal is the inversion at the extremes: the strongest clean model degrades the fastest, retaining only 5.7% at $k = 10$ against RGCN's 23.2%, in the same direction in all three seeds although the three-seed paired test does not reach significance ($p = 0.076$, Section 4.5). The ordering *among* the mid-tier baselines is indicative rather than established, because the spatial depth was selected on RGSAGE-Max and then applied unchanged to every architecture (Section 4.1); a per-architecture depth could plausibly reorder them, and would in particular raise RGATv2, whose high relative retention rests on a clean score depressed by instability at that depth.

This ordering follows from how each operator treats the zeroed embedding a dropped sensor propagates (Section 3.4). The max pooling of Eq. (3) forwards the single largest activation in a neighborhood, so one corrupted message can dominate a live node's update, making RGSAGE-Max the most fragile despite being the strongest clean model, whereas the normalized sum of Eq. (2) and mean pooling average that message against surviving ones and degrade more gracefully. Attention (Eq. (4)) could in principle down-weight a dropped neighbor, but its coefficients are computed from features and are not availability-aware. GRU performs no spatial aggregation, so it is insensitive to neighbors' dropout but capped by the absence of spatial reasoning.

The aggregation operator is therefore a weak lever for robustness: even the most robust baseline retains under a third of its accuracy at 40% dropout. Deployment-level robustness comes from training instead. Dropout-aware training raises retention from 5.7% to 80.3% on the RGSAGE-Max backbone (Table 6) and, altering only the input masking, applies unchanged to every architecture (Table 5, DA columns). SDR-STGNN confirms this from the opposite side: silencing every dropped node at every hop also removes the multi-hop paths that would otherwise route information around a failure, so the gate adds nothing over the augmentation. Practically, on this feeder an operator could lose roughly 10 of 25 μPMUs distributed across the feeder while macro-F1 stays above 0.67, provided the model is trained with the augmentation; losses concentrated near the fault degrade further (Section 4.3). This supports resilient monitoring of distribution feeders without additional sensing hardware.

5.2. *Why Dropout-Aware Training Works*

Separating *that* the augmentation works from *why* matters, because the mechanism determines when it should transfer. Each node's input carries three availability-mask channels alongside its three phase voltages (Section 3.1), and those channels are not trivial: only 14 of the 25 μPMUs are three-phase, so the rest present patterns such as $(1, 0, 0)$. They are, however, entirely *static* — the phase instrumentation of a feeder does not change — so a network trained on that corpus reads the mask as a fixed per-node fingerprint, never as a live availability signal. Dropping a sensor at inference zeroes all six of its channels, so its mask becomes $(0, 0, 0)$, a pattern that node has never once presented during training: the model has no learned response to it and reads the zeroed voltages as a legitimate near-zero measurement, which the max-aggregation argument above explains will then dominate a live node's update. Dropout-aware training is the first thing that makes the mask vary within a node's own history, so $(0, 0, 0)$ acquires the meaning "this neighbor is dark now" and the network learns the down-weighting for itself rather than being handed it by the architecture.

Three of our results are what that account predicts. First, an explicit architectural gate should then be largely redundant, because the model has already learned the function the gate hard-codes — and SDR-STGNN does not exceed the augmentation (ΔAUDC = 0.021, $p = 0.098$; Table 7). Second, supplying plausible numbers should not substitute for the training signal, because the deficit is misplaced trust rather than absent values — and mean-imputation, which

even sets the mask to 1, gains only 0.051 macro-F1 at $k = 10$ ($p = 0.102$). Third, the augmentation can only exploit redundancy the architecture can reach, so its benefit should scale with spatial connectivity — and the AUDC gain is largest on RGSAGE-Max (+0.511) and smallest on the temporal-only GRU (+0.259), which has no message passing with which to route around a dark node. Dropout-aware training does not reconstruct the missing measurement; it trains the model out of depending on any particular one, and the four-hop receptive field over a 25-node tree is what leaves it somewhere else to look.

5.3. Limitations

Several limitations bound these findings. All experiments use the IEEE 123-bus feeder in its default configuration, so generalization across network sizes and topologies, including the reconfigured ("green") topology of [1], remains open (Section 3.3). Our missingness model drops sensors independently and uniformly at random, whereas real μPMU failures are often correlated: co-located units, a shared communication path, or a substation outage fail together, and such structured losses may degrade, and respond to mitigation, differently. Persistence itself is not the gap — holding the dropped set fixed for a whole event leaves every curve unchanged (Section 4.3) — but spatial correlation between the failed units remains unmeasured. The degradation curves are consequently averages over random sensor subsets rather than worst cases; Section 4.3 bounds the worst case by dropping the μPMUs nearest the fault, but only on the RGSAGE-Max backbone and only for proximity-ordered loss, so the other four architectures and other adversarial orderings remain unmeasured. Voltages are also taken directly from the simulator, without additive measurement noise, quantization, or communication latency, so the full-observability accuracies are optimistic relative to field μPMU data; conversely, the three stable spatial GNNs were still improving at the end of the epoch budget, so their accuracies are a floor. Our imputation comparison covers only lightweight graph mean-imputation (Section 4.3), not learned reconstruction such as the Dirichlet/contrastive pipeline of [11] or the VGAE of [6], so whether a stronger imputer could close the gap is unresolved. The cross-architecture orderings of Tables 4 and 5 are indicative rather than firm, for the reasons established in Section 4.5. Finally, dropout-aware training samples $k \in \{0, 1, 2, 5\}$, so the $k = 10$ evaluation probes beyond the training distribution; recovery remains strong there, but we ran no ablation over the training dropout grid, leaving open whether including $k = 10$ during training would raise the $k = 10$ result further or cost additional clean accuracy.

6. Conclusion and Future Work

Building on the STGNN fault-localization methodology of [1], this work contributes a measurement and a negative result. The measurement is that clean accuracy is an unreliable indicator of robustness to μPMU loss: the highest-performing baseline under full observability degraded the fastest as sensors were removed, in absolute as well as relative terms. The negative result is that an availability-gated architecture, SDR-STGNN, achieved comparable but not superior robustness to a purely training-time fix, so on this task the aggregation operator is a weak lever and robustness is better bought during training. That fix, dropout-aware training, raised the retained macro-F1 at 40% sensor dropout from 5.7% to 80.3% on the RGSAGE-Max backbone ($p_{\text{Holm}} = 0.003$) at measured parity in inference latency and parameter count and with no measurable loss of clean accuracy, and improved every architecture tested, although on RGCN and RGSAGE-Mean it cost roughly 0.1 macro-F1 under full observability. Future work will investigate correlated and spatially structured sensor failures, worst-case sensor loss beyond the single backbone measured here, larger and more diverse feeder datasets, cross-topology generalization to reconfigured feeders, adaptive availability-gating mechanisms, and calibrated uncertainty of the kind reported for traffic sensors [14], which would let a model signal when observability has degraded far enough that its prediction should not be acted on.

Acknowledgements

The authors thank the Department of Computer Science and Engineering, Amrita School of Computing, Coimbatore, Amrita Vishwa Vidyapeetham, for the institutional support that made this work possible.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) in order to draft and edit manuscript text, improve readability, and resolve LaTeX formatting. All experimental design, simulation, training, and evaluation were carried out by the authors, and every reported number, table, and figure was produced by the authors' own experimental pipeline. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] Karabulut, B., C. Manna, and C. Develder. (2026) "Robustness of spatio-temporal graph neural networks for fault location in partially observable distribution grids." *arXiv preprint* **arXiv:2604.20403**.

[2] Krishnathevar, R., and E. E. Ngu. (2012) "Generalized impedance-based fault location for distribution systems." *IEEE Transactions on Power Delivery* **27** (1): 449–451.

[3] Liu, F., L. Xie, K. Yu, Y. Wang, X. Zeng, L. Bi, et al. (2023) "A novel fault location method based on traveling wave for multi-branch distribution network." *Electric Power Systems Research* **224**: 109753.

[4] Livani, H., C. Y. Evrenosoglu, and V. A. Centeno. (2013) "A machine learning-based faulty line identification for smart distribution network." *North American Power Symposium (NAPS)*: 1–6.

[5] Chen, K., J. Hu, Y. Zhang, Z. Yu, and J. He. (2020) "Fault location in power distribution systems via deep graph convolutional networks." *IEEE Journal on Selected Areas in Communications* **38** (1): 119–131.

[6] Fan, M., J. Xia, H. Zhang, and X. Zhang. (2024) "Fault location method of distribution network based on VGAE-GraphSAGE." *Processes* **12** (10): 2179.

[7] Nguyen, B. L. H., T. Vu, T.-T. Nguyen, M. Panwar, and R. Hovsapian. (2022) "1-D convolutional graph convolutional networks for fault detection in distributed energy systems." *IEEE 1st Industrial Electronics Society Annual On-Line Conference (ONCON)*: 1–6.

[8] Ngo, Q.-H., B. L. H. Nguyen, J. Zhang, K. Schoder, H. Ginn, and T. Vu. (2025) "Deep graph neural network for fault detection and identification in distribution systems." *Electric Power Systems Research* **247**: 111721.

[9] Sachin, S., V. S. Narayanan, M. S. Trisha, K. H. Reddy, and A. Mahadevan. (2025) "AGNet: an adaptive encoder-decoder GNN for spatio-temporal forecasting." *Proceedings of the International Conference on AI and Robotics (AIR), Lecture Notes in Networks and Systems*: 131–149.

[10] Karabulut, B., C. Manna, and C. Develder. (2025) "Generalization of graph neural network models for distribution grid fault detection." *arXiv preprint* **arXiv:2510.03571**.

[11] Zhang, L., D. Hua, T. Ji, T. Qian, and J. Wang. (2025) "Fault location in power distribution networks with massive missing data: a graph-based imputation and contrastive learning approach." *IEEE Transactions on Automation Science and Engineering* **22**: 16825–16837.

[12] Jayapal, S., and R. Annamalai. (2025) "Graph neural network with Hessian-based locally linear embedding for cancer metastasis analysis in lymph nodes using DeepLab segmentation." *IEEE Access* **13**: 46448–46458.

[13] Asiri, A., and K. Somasundaram. (2025) "Graph convolution network for fraud detection in bitcoin transactions." *Scientific Reports* **15**: 11076.

[14] Kirthan, P., M. Anbazhagan, R. Reddy, and H. Kurunathan. (2026) "Towards uncertainty-calibrated traffic flow prediction using combinatorial graph neural networks." *Proceedings of the 12th International Conference on Vehicle Technology and Intelligent Transport Systems (VEHITS)*: 79–91.

[15] Hamilton, W., Z. Ying, and J. Leskovec. (2017) "Inductive representation learning on large graphs." *Advances in Neural Information Processing Systems* **30**: 1024–1034.

[16] Brody, S., U. Alon, and E. Yahav. (2022) "How attentive are graph attention networks?" *International Conference on Learning Representations (ICLR)*.

[17] Rong, Y., W. Huang, T. Xu, and J. Huang. (2020) "DropEdge: Towards deep graph convolutional networks on node classification." *International Conference on Learning Representations (ICLR)*.